

멀티암 밴딧 (Multi-Armed Bandits)

밴딧 정식화와 ϵ -greedy · 낙관적 초기화와 UCB · 밴딧에서 MDP로

Machine Learning 2 · 2주차

한국과학영재학교 (KSA)

Chapter 2

이번 주차 개요

카지노의 슬롯머신: 각 손잡이(arm)는 **모르는** 평균 보상을 가진 보상 기계. 제한된 횟수만 당길 수 있다면, 어떤 순서로 당겨야 총 보상을 최대화할 수 있을까? — 이 단순한 질문이 **멀티암 밴딧** 문제.

- 상태(state)도, 전이(transition)도 **없다** — **탐험과 활용의 딜레마** 하나만 순수하게 남긴 **가장 작은 뼈대**.
- 다음 장(MDP)에서는 바로 이 밴딧에 “상태”라는 재료 하나를 더해서 수학적으로 정식화한다.

이름의 유래: bandit machine(슬롯머신) + 팔이 여러 개인 **문어**의 말장난.

1930년대 말부터 수학 연구, 1952년 Robbins의 후회 하한($O(\ln T)$)으로 확립.

두 잣대: **총보상** $\sum R_t$
(최대화) · **후회**(오라클과의 차이, 최소화).

이 챕터를 마치면

k 개 팔 밴딧을 **총보상·후회**로 정식화하고,
 ϵ -greedy·낙관적 초기화·UCB를 “탐험이 어떻게 만들어지는가” 기준으로 비교하며, 밴딧을 “상태가 하나뿐인 MDP”로 해석할 수 있다.

세 차시 로드맵

차시	내용
2.1	밴딧 정식화(총보상·후회), ϵ-greedy 구현, 증분 갱신식 = 온라인 평균 증명, ϵ 곡선 실험
2.2	낙관적 초기화·UCB : 탐험을 알고리즘 내부에서 자동 유도, 세 전략 비교, 비정상 환경
2.3	밴딧 = 1-상태 MDP , 2-상태 벨만방정식 손계산, “밴딧인가 MDP인가” 판정 기준, 임상시험·A/B

핵심 질문: “알아보려고 쓰는 시간(탐험)과 지금까지 가장 좋았던 것에 집중하는 시간(활용)을 어떻게 나눌 것인가?”

2.1 탐험과 활용: ϵ -greedy

탐험-활용의 딜레마

- **탐험(exploration)** — 아직 모르는 것에 대해 알아보려고 쓰는 시간.
- **활용(exploitation)** — 지금까지 가장 좋았던 것에 집중하는 시간.
- 슬롯머신: “이 손잡이가 좋은 것 같긴 한데 확신은 못 하겠네”라는 판단 순간부터 매 스텝마다 다시 제시됨.

이 딜레마는 이 책의 **거의 모든 장**에서 다시 등장: Ch. 5(몬테카를로), Ch. 6(TD 학습), Ch. 9~16(딥러닝).

이번 절에서 만들 것

ϵ -greedy 정의·구현, 증분 갱신식의 수학적 정체 확인, “탐험에도 비용이 있다”를 숫자로 만들기.

밴딧 문제의 정적 설정

k 개의 팔(arm), 각 팔 a 는 **모르는** 진짜 평균 보상 $q^*(a)$ 를 가진 “보상 기계”.

- 매 스텝 $t = 1, 2, \dots, T$: 팔 하나를 골라 당기고 보상 R_t 를 받는다.
- $R_t \sim \mathcal{N}(q^*(a_t), 1)$ — 평균 $q^*(a_t)$, 표준편차 1의 정규분포.
- **판별 가능한 신호는 “평균” 하나뿐** — 매번 받는 보상은 잡음에 가려져 있다.

목표

T 스텝 동안의 **총보상** $\sum_{t=1}^T R_t$ 를 **최대화**.

오라클과 후회(regret)

오라클(cheater): 진짜 최선의 팔 $a^* = \arg \max_a q^*(a)$ 를 매 스텝 골라 $T \cdot q^*(a^*)$ 벌어들임.

우리는 $q^*(a)$ 를 몰라서 오라클을 **원칙적으로** 맞추지 못한다. 그러나 “오라클과의 차이”를 잣대로 쓰면 탐험이 얼마나 합리적이었는지를 한 숫자로 압축:

$$\text{후회(regret)} \quad \mathcal{R}_T = T \cdot q^*(a^*) - \sum_{t=1}^T R_t$$

모든 탐험 전략의 궁극적 목표: $\mathcal{R}_T$ 를 **0으로 가까이**하는 것.

왜 “상태가 없는” 문제가 출발점인가

MDP의 복잡성(Ch. 3: “지금 한 행동이 다음 상황을 바꾼다”)를 건어내고 **탐험-활용의 딜레마** **하나만** 남긴 문제. 상태가 없으므로 “정책”은 “지금 Q, N 이 이렇다면 어떤 팔을 고를 것인가”라는 **한 줄 규칙**으로 단순화 — 그 한 줄 규칙이 ϵ -greedy.

탐욕적 선택의 함정

가장 단순한 전략: **탐욕적(greedy) 선택** — 매번 $Q_t(a)$ 가 가장 큰 팔을 당기기.

문제: 초반에 우연히 나쁜 결과가 나온 팔을 **영원히 외면**할 수 있다.

- 1 팔 A가 사실 최선($q^*(A) = 2.0$)인데, 첫 보상이 잡음 때문에 0.1로 낮게 나와 $Q(A) = 0.1$.
- 2 팔 B의 첫 보상이 1.4로 나와 $Q(B) = 1.4 > Q(A)$.
- 3 탐욕 규칙은 이후 **영원히** B를 고른다. $Q(A)$ 가 2.0에 접근할 기회는 **구조적으로** 사라짐.

A를 다시 당기는 확률이 $0 \rightarrow Q(A)$ 는 0.1에 박혀 영원히 “A는 나쁜 팔”이라는 **틀린 확신**이 됨.

구조적 함정: 데이터가 늘수록 더 틀림

- 매 팔의 첫 관찰은 진짜 평균의 **무작위 샘플 하나**. 표준편차 1의 잡음에 한 번의 보상이 평균을 정확히 반영할 리 없음.
- 순수 탐욕은 “추정치가 낮은 팔”을 **한 번도** 다시 골라주지 않음.
- 관찰 데이터가 늘어날수록 **틀린 결론에 대한 확신만 늘어남** — 통계적으로 가장 불쌍한 실패 모드.

현실에서의 이름: 콜드 스타트 (cold start)

추천시스템에서 새로 들어온 영화(팔)는 평점이 하나도 없어서 (또는 우연히 낮은 첫 평점을 받아서) 추천 순위에서 **영영 밀려나는** 현상.

ϵ -greedy 전략 (Sutton & Barto, 2018)

ϵ -greedy — 확률 $1 - \epsilon$ 으로 현재 가장 좋은 팔을 당기고(활용), 확률 ϵ 으로 무작위 팔을 당긴다(탐험).

추정치는 실제로 관찰한 보상의 평균으로 갱신:

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)} (R_t - Q_t(a))$$

$N_t(a)$ = 팔 a 를 지금까지 당긴 횟수.

이 갱신 규칙은 “새로 관찰한 값과 추정치의 차이만큼, 그 크기에 비례해서 옮긴다”는 형태 — Ch. 5(몬테카를로) · Ch. 6(TD 학습)에서 재사용.

선택 확률: 두 극단의 혼합

ε -greedy가 팔 a 를 고를 확률:

$$P(a_t = a) = \begin{cases} (1 - \varepsilon) + \frac{\varepsilon}{k} & \text{if } a \text{가 greedy 팔} \\ \frac{\varepsilon}{k} & \text{그 외} \end{cases}$$

- **첫째:** 어떤 팔이든 선택 확률 $\varepsilon/k \geq 0$ 으로 **0이 아님** — “한 번의 나쁜 운”에 영원히 박힐 구조적 경로가 **수학적으로 막혀**.
- **둘째:** greedy 팔의 확률 $1 - \varepsilon + \varepsilon/k$ 로 나머지 $(k - 1)$ 개와 **불균형** — 정보가 “이 팔이 좋아 보여”라고 하면, 그 판단을 $(1 - \varepsilon)$ 만큼 **존중**.

ε 이 이 전략의 **유일한** 조종장치: 탐험의 “양”을 한 하이퍼파라미터가 완전히 결정.

ϵ -greedy 구현

```
import random

def epsilon_greedy_bandit(true_means, epsilon, steps):
    k = len(true_means)
    Q = [0.0] * k
    N = [0] * k
    for t in range(steps):
        if random.random() < epsilon:
            a = random.randrange(k)          # 탐험
        else:
            a = max(range(k), key=lambda i: Q[i]) # 활용
        r = random.gauss(true_means[a], 1.0) # 잡음섞인보상
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
    return Q
```

손으로 한 번: 증분 갱신식 따라가기

팔 A, B 두 개, $Q_A = Q_B = 0$, $N_A = N_B = 0$ 에서 시작. A, A, B, B 순서로 당기고 보상 2, 0, 3, 1 관찰:

시도	당긴 팔	갱신 계산	갱신 후
1	A	$Q_A \leftarrow 0 + \frac{1}{1}(2 - 0)$	$Q_A=2.0, Q_B=0.0$
2	A	$Q_A \leftarrow 2 + \frac{1}{2}(0 - 2)$	$Q_A=1.0, Q_B=0.0$
3	B	$Q_B \leftarrow 0 + \frac{1}{1}(3 - 0)$	$Q_A=1.0, Q_B=3.0$
4	B	$Q_B \leftarrow 3 + \frac{1}{2}(1 - 3)$	$Q_A=1.0, Q_B=2.0$

$Q_A = 1.0$ 은 A의 보상 2, 0 **평균**, $Q_B = 2.0$ 도 B의 3, 1 **평균**과 정확히 같음. 이 시점에서 탐욕적 선택하면 $Q_B > Q_A$ 이므로 **팔 B** 선택.

증분 갱신식 = 온라인 평균: 귀납으로 증명

팔 하나를 n 번 당겨 관찰한 보상 $R_1, \dots, R_n$ 일 때 ($Q_0 = 0$ 에서 시작):

- **기초 단계** $n = 1$: $Q_1 = 0 + \frac{1}{1}(R_1 - 0) = R_1 = \text{평균}.$
- **귀납 단계**: $Q_n = \frac{1}{n} \sum_{i=1}^n R_i$ 라고 가정하고 한 번 더 갱신:

$$\begin{aligned} Q_{n+1} &= Q_n + \frac{1}{n+1}(R_{n+1} - Q_n) = \frac{n}{n+1}Q_n + \frac{1}{n+1}R_{n+1} \\ &= \frac{1}{n+1} \sum_{i=1}^n R_i + \frac{R_{n+1}}{n+1} = \frac{1}{n+1} \sum_{i=1}^{n+1} R_i \quad \blacksquare \end{aligned}$$

$\frac{1}{n+1}$ 의 가중치가 “이전 평균에 n 번, 새 관측에 1 번” 나눠주는 구조.

이 등치가 왜 중요한가

- ① “학습 규칙”이라 복잡하게 들리던 식 뒤에 **평균**이라는 잘 아는 개념이 숨어 있음 — $Q_t(a)$ 는 “불확실한 마법의 숫자”가 아니라 **그 팔에 대한 현재 최우량 추정치(평균)**.
- ② $\frac{1}{N}$ 의 **학습률**이 시도에 비례해 줄어듦 — 100번째 관찰은 $\frac{1}{100}$ 만 움직이고, 1000번째는 $\frac{1}{1000}$. Ch. 6의 “ $1/N$ vs 고정 α ” 논쟁의 한쪽 극.
- ③ **온라인(online)**: 보상을 전부 기억할 필요 없이 Q, N 두 숫자만으로 전체 평균 갱신 — 수백만 스텝을 돌릴 때 메모리가 $O(k)$ 로 유지되는 이유.

자주 하는 실수: $\varepsilon = 0$ (순수 탐욕) 그대로 쓰기

진짜 평균 [2.0, 1.0] (A가 최선), *random.seed*(11), 200스텝:

$\varepsilon = 0$ (순수 탐욕):

Q = [-0.113, 0.894]
N = [1, 199]

팔 A를 **딱 한 번**만 시도 → 이후 199번 내내 B만 당김. “확신”해버린 뒤로는 재시도 불가.

$\varepsilon = 0.1$ (탐험 10%):

Q = [1.974, 0.739]
N = [183, 17]

진짜 최선인 A를 183번 시도 → 추정치(1.974)가 진짜 평균(2.0)에 근접.

핵심

$\varepsilon = 0$ 은 이론상 “효율적 활용만”처럼 보이나, 실전에서는 **단 한 번의 나쁜 운으로 최적해를 영원히 놓칠 수 있는** 가장 위험한 선택.

연관 실수 “첫 스텝의 숨겨진 임의성”: $t = 1$ 에는 모든 $Q(a) = 0$ (완전 동점)이라 $\max(\dots)$ 가 **첫 팔**(인덱스 0) 반환 — $\varepsilon = 0$ 이면 첫 당김이 알고리즘이 정해버리는 숨겨진 편향. $\varepsilon > 0$ 이면 나머지 팔이 계속 재시험되어 무해; $\varepsilon = 0$ 이면 이 첫 운이 **결정적** 결과(= 위 seed=11 예).

역사: “밴딧”이라는 이름, 70년 된 문제

- 이름: 카지노 **슬롯머신**(구식: bandit machine).
- 1930년대 말부터 수학 연구 시작.
- **1952, Robbins**: “어떤 전략이든 후회가 $O(\ln T)$ 이하가 될 수 없다”는 **하한** 증명 — “탐험을 **최소한** 이만큼은 해야 한다”.
- 이 하한에 도달하는 전략: 2.2절의 **UCB**.
- ϵ -greedy는 “가장 단순한” 전략이자, 70년 넘게 연구된 문제의 **출발점**.

실제 적용 분야: A/B 테스트(적응형), 뉴스 기사 추천, 임상시험(2.3절), 온라인 광고 배너 선택.
공통점: “한 번의 선택이 다음 선택지를 바꾸지 않고” 매 스텝 같은 후보 집합에서 다시 고르는 구조 = 상태 1개 MDP.

확인 문제: 선택 확률 직접 계산

ε -greedy의 $P(a_t = a)$ 식을 써먹자:

문항	설정	답
(a)	$k = 5$, $\varepsilon = 0.2$, A가 greedy	A: 0.84, B·C·D·E: 각각 0.04
(b)	같은 설정, B가 사실 A보다 좋은 팔	B 선택 확률 = 0.04 = 4%
(c)	$\varepsilon = 0$	출구 확률 = 0 — 구조적으로 출구 단힘

(b): $T = 2000$ 스텝이면 B는 기대 $2000 \times 0.04 = 80$ 번 재시행됨.

한 줄 요약

은 “틀린 확신에서 탈출할 출구의 폭”을 결정하는 숫자.

탐험에도 비용이 있다

ϵ -greedy가 무작위 팔을 고르는 스텝에서, 그 “무작위 팔”의 기대 보상 = k 개 팔의 평균 $\frac{1}{k} \sum_a q^*(a)$ — 최선의 팔 $q^*(a^*)$ 보다 **작다**.

탐험 스텝 하나당 기대 손실:

$$q^*(a^*) - \frac{1}{k} \sum_a q^*(a)$$

$k = 3, q^* = [1.0, 1.5, 2.0]$ 이면 스텝마다 $2.0 - 1.5 = \mathbf{0.5}$ 의 기대 손실.

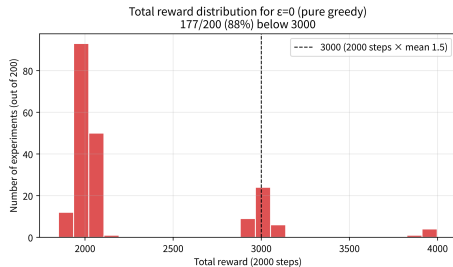
- ϵ **너무 작으면**: “한 번의 나쁜 운”에 최적 팔을 놓칠 확률 증가.
- ϵ **너무 크면**: 이미 찾은 최선 팔을 외면한 채 스텝마다 0.5 손실 지속.
- 양쪽 끝이 나쁘면 **가운데 어딘가에** 최대치 존재.

실험: ϵ 을 0 $\rightarrow$ 1로 옮기

실험 설정: $k = 3$, $q^* = [1.0, 1.5, 2.0]$, $T = 2000$, $\epsilon \in \{0, 0.01, 0.05, 0.1, 0.2, 0.5\}$, 각 ϵ 마다 200번 독립 실험(시드 1000~1199).

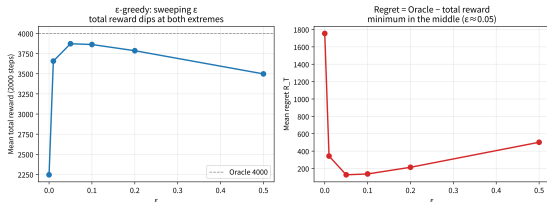
ϵ	평균 총보상	평균 후회	최선편에 몰림
0.00	2245.6	1754.4	3% (97% 막힘)
0.01	3657.1	342.9	84%
0.05	3872.4	127.6	98%
0.10	3862.8	137.2	100%
0.20	3785.9	214.1	100%
0.50	3498.1	501.9	100%

“최선편에 몰림” = 종료 시 최선편 팔 C가 50% 이상 선택된 실험 비율.
 $\epsilon = 0$ 은 97% 실험이 “막힘”.



$\epsilon = 0$ 총보상 히스토그램: 97% 실패 vs 3% 성공 — “가끔은 잘 동작한다”는 착시를 깨는 양분포.

ϵ 곡선: 최대치는 “가운데”



왼쪽: 평균 총보상, 오른쪽: 평균 후회. $\epsilon = 0$ 은 후회 1754로 폭주, $\epsilon = 0.5$ 는 502. 최대치는 $\epsilon \approx 0.05 \sim 0.1$.

세 가지 관찰:

- 1 $\epsilon = 0$ 의 후회 **1754** — “단 한 번의 나쁜 운”이 약 2000스텝 $\times$ 2.0의 절반에 달하는 손실. $\epsilon = 0.1$ 만 달이면 후회 ≈ 137 로 **약 13배** 감소.
- 2 **최대가** $\epsilon = 0.05$ (3872.4). 0.10(3862.8)은 “아주 비슷”한 2위. “ $\epsilon = 0.1$ 이 마법 수”가 아니라 **문제에 따라 움직이는** 절절점.
- 3 $\epsilon = 0.01$ **도 16% 실패** — “좀만 섞어주면 되겠지”가 “영원히 박히기”를 **완전히** 막아주진 못함.

후회 137을 “예측”하기: 손실 분해

$\varepsilon = 0.1$ 이면 $2000 \times 0.1 = 200$ 스텝을 무작위 팔에 사용.

$$\text{순수 탐험 손실} = 200 \times 0.5 = \mathbf{100}$$

$$\text{초기 불확실성 손실} \approx 137 - 100 = \mathbf{37}$$

- **탐험 손실(100):** “활용을 포기한 대가” — 무작위 팔에 쓴 스텝마다 기대 0.5 손실.
- **초기 불확실성 손실(37):** “최선 팔을 **아직 못 찾은** 초반”의 손실 — 추정치가 불신실한 초기 구간에서 greedy 선택도 가끔 오판.

곡선의 모양

ε 줄이면 “탐험 손실”은 줄지만 “초기 불확실성 손실”은 커짐. 두 손실의 합이 **최소**가 되는 지점 = 곡선의 꼭대기.

자주 묻는 질문 (1): ϵ 를 크게?

Q: ϵ 을 크게 잡을수록 항상 더 좋은가?

아니다.

- ϵ 이 0이면 최적해 영영 놓칠 위험.
- 지나치게 크면(예: 0.5) 이미 최선의 팔을 찾은 뒤에도 나쁜 팔을 절반 확률로 계속 시도.
- 실전: 학습 진행될수록 ϵ 을 점점 줄이는 **decay** 가 널리 쓰이는 절절점.

Q: $\frac{1}{N_t(a)}$ 대신 고정 α 를 쓰면?

- $\frac{1}{N}$: 시도 횟수 $\uparrow \rightarrow$ 갱신 폭 $\downarrow \rightarrow$ 정확한 평균으로 **수렴**.
- 고정 α : 정확히 수렴하지 않으나, **최근 관찰**에 더 큰 가중치를 계속 유지.
비정상(non-stationary) 환경에서 오히려 유리.

자주 묻는 질문 (2): $\epsilon=0.1$ 마법 수? decay?

Q: $\epsilon = 0.1$ 이 “마법 수”인가?

아니다. 최대는 **문제마다 다른 위치**. k 가 커질수록(수천 개 광고 배너) 무작위 팔이 **나쁜 팔에 떨어질 확률** $(k-1)/k$ 로 높아져 $\epsilon = 0.1$ 탐험이 거의 전부 낭비. k 큰 문제 $\rightarrow$ **UCB**처럼 불확실성에 비례해 조절하는 전략 유리.

Q: decay는 구체적으로 어떻게?

- $\epsilon_t = \epsilon_0/t$ — 매우 공격적 감쇠.
- $\epsilon_t = \max(\epsilon_{\min}, \epsilon_0/\sqrt{t})$ — 느린 감쇠 + 하한. $\epsilon_{\min} > 0$ 은 비정상 환경 대비.

핵심: ϵ 을 “**학습의 진행도**”에 비례해 줄임.

2.1절 정리

핵심

ϵ -greedy는 “탐험의 양을 하나의 확률 ϵ 으로 조종한다”는, 이 책에서 첫 번째로 배우는 정책.

- $\epsilon = 0$: “한 번의 나쁜 운”에 최적해 영원히 잃을 수 있음(2000스텝 후회 1754).
- ϵ 지나치면: 이미 찾은 최선 팔을 외면한 채 보상 낭비(후회 502).
- **탐험에도 비용이 있다** — 그 비용과 “탐험 부족”의 대가를 줄이면 **가운데 어딘가에** 최적 ϵ .

이 “탐험의 비용” 감각은 2.2절(UCB·낙관적 초기화)에서 **불확실성에 비례해** 탐험을 조절하는 방식으로 정교화되고, Ch. 5(몬테카를로)에서 **후회(regret)**로 정식 정의.

2.2 낙관적 초기화와 UCB: 불확실성을 이용한 탐험

두 가지 철학: “내부 유도형” 탐험

2.1절의 ϵ -greedy는 탐험 비율을 **정해진 확률** ϵ 로 정하는 “외부에서 조종하는” 방식.

이 절의 두 전략은 정반대 철학 — **탐험을 알고리즘 내부에서 자동으로 유도:**

- **낙관적 초기화(optimistic initialization):** “모든 팔을 처음엔 과대평가한다”는 **시작값 하나로** 순수 탐욕만으로도 초반 탐험이 자동 생성.
- **UCB(Upper Confidence Bound):** “아직 덜 당겨본 팔의 추정치는 그만큼 불확실하다”는 점을 **매 스텝마다** 계산에 반영.

왜 “내부 유도형”이 필요한가? ϵ -greedy의 탐험은 정보와 무관하게 발생 — 이미 “최악”이라 확신한 팔도 계속 당김. 이 절의 전략은 탐험을 불확실성에 비례해서만 씀.

낙관적 초기화: 시작값 하나로 탐험을 유도

모든 $Q_0(a)$ 를 실제 가능한 보상보다 **훨씬 크게**(낙관적으로) 초기화.

- 아직 안 당겨본 팔은 “과대평가된” 상태 → 순수 **탐욕적 선택만**으로도 자연스럽게 먼저 시도.
- 시도해서 실망하면(실제 보상 < 초기값) 추정치가 내려가고, 그다음 팔로 넘어감.
- **초반 몇 번의 강제적 탐험을 낙관적 시작값 하나로 만들어내는 셈.**

왜 “크게” 잡아야 하는가

작은 값(예: 0)으로 초기화하면 증분 갱신식이 추정치를 **올리는 방향**으로만 움직임 → 어떤 팔이든 “0보다 낮네”로 판정받은 순간 나머지 팔이 **영원히 외면**될 수 있음. **과대평가(상단에서 시작)**만이 “기회는 실망하면 회수한다”는 올바른 방향의 탐험을 만들.

손으로 한 번: 3팔 밴딧, 초기값 5.0

팔 A, B, C, 초기값 $Q_0 = 5.0$, 실제 평균 $[1.0, 1.5, 2.0]$
(매번 $\sigma = 1$ 잡음).

스텝	선택(탐욕)	보상 R	갱신 후 (Q_A, Q_B, Q_C)
1	A (동점→첫 팔)	0.64	(0.64, 5.00, 5.00)
2	B (B, C가 5.0)	0.94	(0.64, 0.94, 5.00)
3	C (C만 5.0)	1.33	(0.64, 0.94, 1.33)
4	C (1.33 최대)	2.47	(0.64, 0.94, 1.90)
5	C	1.35	(0.64, 0.94, 1.72)

핵심 = 스텝 1~3: 탐험 확률 0(매 스텝 $\max Q$ 만 고르는
순수 탐욕)인데도 $A \rightarrow B \rightarrow C$ 순서로 **모든 팔이 정확히 한
번씩** 당겨짐.

메커니즘: “가짜 믿음”

- 초기값 5.0이 “**안 당겨본 팔은 5.0짜리 팔**”이라는 가짜 믿음을 만들
— 실제 보상(2.0 근처)보다 위에
있어, 추정치가 2.0 아래로 떨어지는
순간 “다음으로 낙관적인 팔”이
자동으로 다음 선택.
- 스텝 4부터는 **진짜 탐욕적 경쟁** —
 Q_C 가 2.5까지 오르면 B(0.94)는
한동안 선택 불가.

한계

낙관적 초기화의 “탐험”은 초기값이 실제
보상분포를 벗어날 때까지만 지속.

UCB: 불확실성 자체를 이용한다

UCB — “지금까지 별로 안 당겨본 팔일수록, 그 추정치가 얼마나 정확한지 자체가 불확실하다”는 점을 직접 활용. 각 팔 고를 때, 평균 추정치에 “**불확실성 보너스**”를 더해 비교:

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- $N_t(a)$ 가 작을수록(적게 당겨봤을수록) 제공된 항 커져 **보너스 커짐** — 그 팔이 사실 최선일 수도 있다는 가능성에 정당한 만큼의 기회.
- 많이 당겨볼수록 보너스 줄어 → 결국 진짜 평균 $Q_t(a)$ 의 크기가 선택을 지배.

UCB 구현: “안 당겨본 팔부터”가 필수

```
import math

def ucb_bandit(true_means, c, steps):
    k = len(true_means)
    Q = [0.0] * k
    N = [0] * k
    for t in range(1, steps + 1):
        unplayed = [i for i in range(k) if N[i] == 0]
        if unplayed:
            a = unplayed[0] # 아직안당겨본팔을먼저시도
        else:
            a = max(range(k), key=lambda i: Q[i] + c * math.sqrt(math.log(t) / N[i]))
        r = random.gauss(true_means[a], 1.0)
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
    return Q
```

unplayed 분기를 빼먹으면 $N(a) = 0$ 대입으로 **0으로 나누기 오류** — 다음 슬라이드.

UCB 공식, 세 조각으로 분해

(a) $Q_t(a)$ — “지금까지의 증거 (utilization)”: 많이 당겨보며 쌓아온 평균 추정치.

(b) $\sqrt{\ln t}$ — 시간. 느리게 증가 → “탐험을 점점 줄이되 **완전히** 끊지는 않는다” 절절점 (t 자체 쓰면 탐험 너무 강해짐).

(c) $\sqrt{1/N_t(a)}$ — 표본 수에 따른 추정치 변동. N 이 4배가 되면 보너스는 정확히 **절반** ($\sigma/\sqrt{n}$ 의 법칙).

역사적 유래: 신뢰구간

“평균 μ 의 $(1 - \delta)$ -신뢰구간 상한 $\approx \hat{\mu} + \sigma\sqrt{2\ln(1/\delta)/n}$ ”이라는 표준 결과를, $\delta = 1/t$ 로 정하면 정확히 UCB 모양. 즉 UCB는 “**각 팔이 얼마나 좋을 가능성(상한)**”을 최대화하는 팔을 고르는 원칙.

손으로 한 번: 보너스 크기 계산

$t = 100, c = 2$ 인 시점에서, 한 팔은 $N = 5$ 번, 다른 팔은 $N = 50$ 번 당겨봤다면:

$$N=5 : \quad 2\sqrt{\frac{\ln 100}{5}} = 2\sqrt{\frac{4.605}{5}} \approx 2 \times 0.960 \approx \mathbf{1.919}$$

$$N=50 : \quad 2\sqrt{\frac{\ln 100}{50}} = 2\sqrt{\frac{4.605}{50}} \approx 2 \times 0.303 \approx \mathbf{0.607}$$

5번밖에 안 당겨본 팔의 보너스(약 1.92)가 50번 당겨본 팔 (약 0.61)보다 **3배 넘게** 큼.

두 팔의 평균 추정치 $Q_t(a)$ 가 비슷하다면, UCB는 이 보너스 차이 때문에 “**덜 당겨본**” 팔을 우선 골라 더 확인하려 함. N 이 커지면 보너스 $\rightarrow 0$, 결국 $Q_t(a)$ 가 지배.

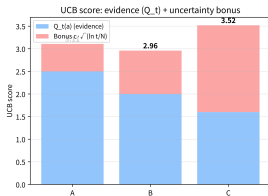
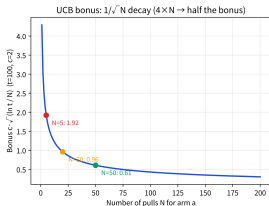
UCB 점수가 선택을 뒤집는지: 3팔 예

같은 시점 ($t = 100, c = 2$), 추정치가 서로 다른 세 팔:

팔	$Q_t(a)$	$N_t(a)$	보너스	UCB 점수
A	2.50	50	0.607	3.107
B	2.00	20	0.960	2.960
C	1.60	5	1.919	3.519

- 순수 탐욕(Q_t 만)이면 A(2.50)가 1위.
- UCB 점수: **C(3.52) > A(3.11) > B(2.96)** — 추정치 **최하**(1.60)인데도 5번밖에 안 당겨본 C가 1위.
- A vs B: A의 추정치가 0.50 높지만, B가 덜 당겨봐서 보너스 0.35 더 큼 $\rightarrow 0.50 > 0.35$ 이므로 **A가 여전히 앞선다**(활용이 이김).

UCB = 연속적인 절절점



왼쪽: 보너스가 $1/\sqrt{N}$ 로 감소. 오른쪽: 추정치
최하(1.60)인 C가 보너스 1.92 덕분에 1위(3.52), 순수
탐욕 1위 A(3.11)를 앞섬.

한눈에 들어오는 구조:

- Q_t 차이(예: A-B의 0.50)가 **보너스 차이 (0.35)보다 크면** → UCB도 순수 탐욕과 같은 팔을 고름(활용이 이김).
- Q_t 차이가 **보너스 차이보다 작으면** → “덜 당겨본” 팔을 고름(탐험이 이김). C vs A: Q_t 차 A가 0.90 앞, 보너스 차 C가 1.31 앞 → **탐험 이김**.

UCB는 “탐험을 0이나 1로만 하는” 것이 아니라, 매 스텝 보너스와 추정치 차이를 비교하는 연속적인 절절점. N 이 커지면 보너스가 $1/\sqrt{N}$ 감소 → 자연스럽게 Q_t 지배 영역으로.

자주 하는 실수: $N(a) = 0$ 을 UCB 공식에 그대로 대입

- $N(a) = 0$ 을 $c\sqrt{\ln t / N(a)}$ 에 대입 $\rightarrow$ **0으로 나누기 오류.**
- 수학적으로도 “한 번도 안 당겨본 팔의 불확실성”은 **무한대에 가까워야** 자연스러움.
- 이걸 놓치고 공식부터 적용하면 **첫 스텝부터 에러**로 멈춤.

초기화 단계가 절대 빠져서는 안 되는 이유

“아직 아무도 안 당겨본 팔이 남아있으면 그것부터 시도한다”는 단계. 이 초기화 자체가 사실 “모든 팔의 불확실성 보너스가 **무한대**인 상태에서 시작해서, 당겨볼수록 유한한 값으로 줄어든다”는 UCB 논리의 자연스러운 **확장**.

확인 문제: “탐험”은 어디서 오는가

기준: “탐험이 정보(지금까지의 Q, N)에 의존하는가, 아니면 그냥 고정 확률인가?”

	전략	분류
(a)	매 스텝 $\varepsilon=0.1$ 무작위, 0.9로 최대 Q	외부 조종형(ε-greedy) — 정보와 무관하게 고정 확률. “최악” 확신한 팔도 계속 당김
(b)	$Q_0 = 5.0$ 초기화, 이후 매 스텝 $\max Q$	내부 유도형(낙관적 초기화) — 초기값이 실제 보상 위에 있어 추정치 내려올 때까지. 이후 추가 장치 없음
(c)	매 스텝 $Q_t + c\sqrt{\ln t / N_t}$	내부 유도형(UCB)

자주 묻는 질문 (1): c 는 어떻게 정하나?

c 가 클수록 → 보너스 커짐 → 더 적극적인 탐험

- c 가 0에 가까울수록 순수 탐욕에 가까워짐.
- Ch. 5.2의 SVM C , Ch. 6의 λ 와 마찬가지로 **검증(또는 시뮬레이션) 성능으로 튜닝**해야 하는 하이퍼파라미터.
- 이론적으로 후회 최소화하는 c 의 범위가 증명되어 있으나(이 책 범위 밖), 실전에서는 $c = 1$ **이나** $c = 2$ **근처**에서 시작해 조정.

보너스 스케일과의 일치

보너스 스케일은 **보상의 변동성(표준편차 σ)에 비례해야** 자연스러움. 이 책 예는 $\sigma = 1$ 이라 $c = 1, 2$ 가 “보너스가 보상 잡음과 같은 스케일”. 실제 $\sigma = 10$ 이라면 $c = 2$ 의 보너스는 잡음에 비해 너무 작아 **탐험이 거의 안 일어남** — c 를 **보상 스케일에 맞춰** 조정해야 한다는 뜻.

자주 묻는 질문 (2): 낙관적 초기화 vs UCB

낙관적 초기화

- 구현이 **훨씬 간단**(초기값만 크게 잡으면 끝).
- 초반 탐험 끝나면 **더 이상 탐험 유도 장치가 없음** — 비정상 환경에서 취약.

UCB

- 매 스텝마다 불확실성을 **다시 계산** → 계속 적응적 탐험.
- 그만큼 **계산이 조금 더 필요**.

세 전략, 한 줄씩

ϵ -greedy: 탐험의 **양**을 확률로 조종 — 간단하지만 정보와 무관하게 탐험.

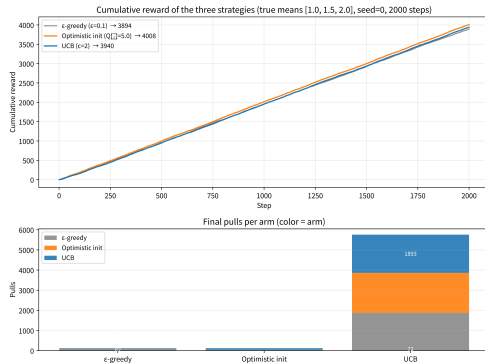
낙관적 초기화: 탐험을 **초기값** 하나로 예약 — 구현은 가장 간단, 초기값이 실제 보상을 벗어날 때까지만.

UCB: 탐험을 **매 스텝의 불확실성**에 비례해 자동 계산 — 가장 정교, 비정상 환경에도 적응.

실습: 세 전략을 같은 문제에서 붙여 싸우기

- 같은 밴딧 문제에서 세 전략(ϵ -greedy, 낙관적 초기화, UCB) 비교.
- “추정치가 얼마나 정확한가”가 아니라 “**그 과정에서 실제로 얼마나 많은 보상을 벌었는가**” — 탐험 자체에도 비용(나쁜 팔을 시도하는 동안 그만큼 보상 손해).
- 각 전략마다 `random.seed(0)` 를 다시 걸면 세 전략이 **같은 무작위 시퀀스를 독립적으로 경험** → **공정한 비교**.

설정: $k = 3$, $q^* = [1.0, 1.5, 2.0]$, 2000스텝. ϵ -greedy($\epsilon = 0.1$), 낙관적 ($Q_0 = 5.0$), UCB($c = 2.0$).



위: 누적 보상 곡선. 아래: 각 팔 당김 횟수 — ϵ -greedy만 나쁜 팔(A, B)을 대량으로 다시 당김.

seed=0, $k = 3$, 2000스텝: 실전 숫자

전략	총 보상	$N = (N_A, N_B, N_C)$	$Q = (Q_A, Q_B, Q_C)$
ϵ -greedy ($\epsilon=0.1$)	3893.9	(82, 50, 1868)	(0.978, 1.315, 2.006)
낙관적 초기화 ($Q_0=5.0$)	4007.7	(3, 1, 1996)	(1.099, 0.103, 2.006)
UCB ($c=2.0$)	3940.2	(35, 72, 1893)	(1.150, 1.476, 2.004)

세 가지 관찰:

- 1 **낙관적 초기화 1위**(4007.7) — 초기값 5.0이 실제 보상(2.0)보다 위에 있어, 각 팔이 **최소 한 번**은 당겨진 뒤 최선의 팔(C)에 집중. 효율 최고.
- 2 **ϵ -greedy 3위**(3893.9) — 정보와 무관하게 나쁜 팔(A: 82회, B: 50회)을 계속 다시 당기며 낭비.
- 3 **UCB 중간**(3940.2) — “불확실성이 큰 팔”만 골라 재확인. ϵ -greedy보다 낭비 적지만, “각 팔 정확히 한 번씩” 최소 비용 보장은 못 해줌.

팔 개수가 늘어나면: $k = 10$ 예

$k = 10$: 평균이 0.5부터 1.3까지 0.1 간격 9개 + 마지막 팔만 2.0. 같은 seed=0, 2000스텝.

낙관적 초기화: 각 팔 **정확히 1번**만 당겨진 뒤
가장 좋게 나온 10번째 팔(평균 2.0)에 집중.
10번째 팔 1991회, 나머지 9개 각 1회. **총 보상**
4001.3.

UCB: “불확실성이 큰 팔”을 계속 골라 재확인.
10번째 팔 1786회, 나머지 10~53회(합계
214회). **총 보상 3802.1.**

- ϵ -greedy의 낭비는 k 에 선형으로 증가 — 무작위 탐험이 나쁜 9개 팔에 떨어질 확률 높음.
- **낙관적 초기화**: “각 팔 1회” 최소 비용, 탐색 비용이 k 에 그침(여기서 9회).
- **UCB**: k 개 모두에 기회를 주되 N 이 커지는 팔은 줄임 — 탐험 비용을 k 에 대해 **서브선형**으로 억제.

팔이 수천 개(온라인 광고 수천 개 배너)인 실전 문제에서는 바로 이 차이가 전략 선택을 가름.

비정상 환경: 환경이 바뀌면 누가 살아남나

비정상(non-stationary) 환경: 진짜 평균 $q^*(a)$ 가 **시간에 따라** 바뀌는 환경 — 카지노 운영자가 중간에 “당첨 확률”을 몰래 바꿔놓는 경우.

시나리오: $k = 3$. 처음 1500스텝 $q^* = [1.0, 1.5, 2.0]$ (C 최선) $\rightarrow$ 뒤 500스텝 $q^* = [0.5, 2.0, 1.0]$ (B가 최선, C는 중간으로 하락).

같은 seed=0, 2000스텝, **마지막 500스텝의 평균 보상** 비교:

비정상 환경 결과: UCB만이 적응

전략	마지막 500스텝 평균	마지막 Q	마지막 N
ϵ -greedy ($\epsilon=0.1$)	+0.97	(0.89, 1.41, 1.75)	(82, 50, 1868)
낙관적 초기화 ($Q_0=5.0$)	+1.02	(1.10, 0.10, 1.76)	(3, 1, 1996)
UCB ($c=2.0$)	+1.86	(1.13, 1.91, 1.96)	(34, 492, 1474)

- **UCB: 1.86**으로 거의 최선(2.0) — C의 보상이 1.0으로 떨어지자, B의 보너스($N_B = 492 < N_C = 1474$)가 여전히 크고, B의 Q 가 1.91까지 올라오며 다시 **활용**으로.
- **낙관적 초기화: 1.02로 새 최선을 놓침** — 초기값 5.0이 “모든 팔 한 번”을 보장한 뒤에는 **추가 탐험 장치가 없어**, 예전 최선 C(1996회)에 계속 매달림. B는 $N_B = 1$ 로 $Q_B = 0.10$ **완전히 틀린 추정치** 유지.
- **ϵ -greedy: 0.97** — 고정 확률 0.1이 “환경이 바뀌었는지 확인하는” 약한 탐험 역할.

핵심 구분: “일괄 예약” vs “매 스텝 재계산”

낙관적 초기화의 “탐험”

- 초반에 일괄 예약(각 팔 1회).
- 이후 추가 탐험 장치 없음.
- 고정(stationary) 환경: 최선 팔 찾기에 성공.

UCB의 “탐험”

- 매 스텝 재계산(N 이 작은 팔에 계속 보너스 부여).
- 비정상 환경: 환경 변화를 감지하고 적응하는 사실상 유일한 전략.

Ch. 6(TD 학습)의 “고정 α vs $1/N$ ” 논쟁과 같은 구조

정보의 “신선도”를 유지하는지가 비정상 환경에서 성패를 가름.

자주 하는 실수: UCB를 낙관적 초기화로 대체

낙관적 초기화와 UCB는 **초반에는** 거의 같은 일을 하는 것처럼 보이나(둘 다 “안 당겨본 팔”을 먼저 시도), **초반 이후가 완전히 다름**.

- ① “초반 k 회만 무작위로 당기고, 이후엔 순수 탐욕”을 UCB로 착각 — 탐험을 k 회로 고정. k 이후 환경이 바뀌어도 **전혀** 적응하지 못함. “각 팔 1회” 보장조차 안 됨.
- ② UCB 보너스를 “한 번만 계산”하고 이후에는 Q_t 만 비교 — N 이 커져도 보너스가 **감소하지 않아**, “탐험이 점점 줄어든다”는 메커니즘이 **완전히 사라짐**. “불확실성 보너스까지 붙인 고정 탐험”이 됨.

UCB 구현의 본질

매 스텝마다 갱신된 $N_t(a)$ 로 보너스를 재계산 해야 함. 이 “재계산”이 UCB의 본질이며, 빼먹으면 UCB가 아님.

2.2절 정리

핵심

낙관적 초기화는 “탐험을 초기값 하나로 예약”하고, **UCB**는 “탐험을 매 스텝의 불확실성으로 재계산” 한다.

- 고정 환경: 둘 다 “최선 팔 찾기”에 성공.
- 환경이 바뀌는 문제에서는 **UCB의 “계속 재계산”만이 살아남음.**
- 두 전략 모두 ϵ -greedy의 “정보와 무관한 고정 탐험”보다 나은 방향 — **불확실성에 비례하는 탐험.**

2.2절 핵심 질문: 같은 문제로 세 전략을 붙여 싸우면, 정보와 무관하게 탐험하는 ϵ -greedy가 가장 **낭비**하고, 팔의 순위가 뒤바뀌는 비정상 환경에서는 매 스텝 보너스를 재계산하는 **UCB만이** 새 최선을 다시 찾아 적응.

2.3 밴딧 vs 완전한 MDP: 다음 장으로 가는 다리

밴딧에는 “상태”가 없다

- 팔을 당겨도 “다음 상태”로 넘어가지 **않음**.
- 매번 같은 k 개의 팔 중에서 고르는, 시간이 흘러도 상황이 **전혀 안 바뀌는** 세계.

Ch. 3의 **MDP(Markov Decision Process)**는 여기에 결정적인 것 하나를 추가:

“지금 한 행동이 다음에 어떤 상태를 보게 될지에 영향을 준다.”

슬롯머신 vs 체스: 체스에서는 지금 둔 수가 **판의 배치(상태)** 를 바꾸고, 그 배치가 다음에 무엇을 할 수 있는지를 결정.

밴딧에서 배운 탐험-활용 딜레마는 그대로 남지만, “지금 선택이 **미래의 선택지 자체**를 바꾼다”는 완전히 새로운 문제가 겹쳐짐.

이 절에서 두 가지를 만든다

- ① **밴딧을 MDP의 언어로 다시 쓰기** — “상태가 딱 하나뿐인 MDP”라는 관점에서 2.1~2.2절의 모든 전략이 **그대로 살아남**는 것을 보임.
- ② **MDP가 실제로 무엇이 추가되는지** — 상태 전이 $P(s'|s, a)$ 가 도입되는 순간, “가장 좋은 팔을 고르는 것”이 아니라 “가장 좋은 **경로(policy)**를 고르는 것”으로 문제가 커지는 것을, **두 상태짜리 장난감 예제**로 직접 계산.

이 둘을 다 지나면 Ch. 3의 다섯 요소 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 가 “어떤 새로운 문제를 해결하기 위해 존재하는지”가 선명해짐 — **밴딧이라는 단순한 출발점 위에서 무엇이 “추가”로 생겼는지를 보는 것.**

유명한 사례 (1): 임상시험 속의 밴딤

전통적 임상시험: 환자를 절반씩 무작위로 신약/위약에 배정하고, 시험 끝날 때까지 비율을 **바꾸지 않음** — 통계적으로 깔끔하지만, 시험 도중 신약이 확실히 더 효과적이라는 증거가 쌓이고 있으면도 계속 절반을 위약에 배정하는 것은 **윤리적으로 부담스러움**.

적응적 임상시험 (adaptive clinical trial)

이 절의 UCB나 낙관적 초기화와 같은 원리로, **중간 결과를 보며 더 유망해 보이는 치료법 쪽으로 배정 비율을 점차 옮김** — “지금까지의 증거로 가장 좋아 보이는 팔에 활용을 더 쏟되, 아직 불확실한 팔도 완전히 버리지 않는다”는 이 장의 핵심 원리가 **그대로 적용**.

유명한 사례 (2): A/B 테스트, 그리고 판정 질문

- **A/B 테스트**: 두 광고 문구 중 어느 쪽이 클릭률이 높은지 확인하는 동안에도 트래픽을 영원히 절반씩 나누는 대신, 밴딧 알고리즘으로 **성과가 좋은 쪽에 점점 더 많은 사용자를 자동 몰아줌**.
- “multi-armed bandit testing”이라는 이름으로 여러 A/B 테스트 플랫폼이 이 기능을 제공.

그런데 이 두 사례는 정말 순수한 밴딧인가?

“어떤 치료법을 쬘지가 **다음에 그 환자에게 쓸 수 있는 치료법 후보 자체**를 바꾸지 않는다”고 가정하면 → 매번 같은 (신약, 위약) 두 후보에서 다시 고르는 → **밴딧으로 모델링 가능**.

그런데 현실: 신약을 주었더니 부작용으로 위약은 더 이상 안 되는 상황이 생기고, 첫 주치의의 선택이 두 번째 주치의의 선택지를 제약 → 행동이 “다음 상황(상태)”을 바꿈 → 그 문제는 **MDP**.

밴딧은 “상태가 하나뿐인 MDP”다

Ch. 3의 MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 에서 $\mathcal{S}$ 의 크기를 **1**로 줄여보자:

- $\mathcal{S} = \{s^*\}$: 상태는 유일.
- 행동 a 를 하면 **다시 같은 상태** s^* 로 돌아옴 — $P(s^*|s^*, a) = 1$, 즉 **자기 전이(self-loop)**만 존재.
- 보상은 $R(s^*, a)$ (밴딧의 $q^*(a)$ 와 정확히 같음).

이 1-상태 MDP에서 에이전트가 할 수 있는 일은 매 스텝마다 k 개 행동 중 하나를 고르는 것뿐 — **밴딧**.

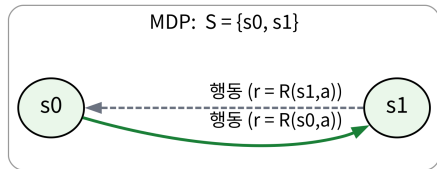
관점 전환

밴딧은 MDP를 정의하고, 그 상태의 개수를 1로 고정한 특수한 경우. 2.1~2.2절의 전략들은 “1-상태 MDP의 정책(policy)”으로 다시 해석됨.

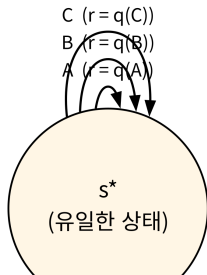
그림으로 보는 “추가 요소” 하나

밴딧 = 1-상태 MDP

2-상태 MDP



밴딧: $S = \{s^*\}$ (팔 = 행동 A, B, C)



- **왼쪽(밴딧):** 상태 s^* 하나 위에 세 개의 **자기 전이**(팔 A, B, C)만 — 어디로도 “넘어가지” 않으므로, 지금 선택이 미래 선택지에 영향을 줄 **통로가 전혀 없음**.
- **오른쪽(MDP):** 상태 두 개(s_0, s_1) — s_0 에서 한 행동이 s_1 로 전이하고(실선), s_1 에서 다시 s_0 로 돌아옴(점선).
- 이 “다른 상태로 넘어가는 실선”이 밴딧에는 없는 유일한 추가 요소 — Ch. 3의 $P(s'|s, a)$.

실용적 효용: “같은 코드”가 Ch. 6까지 살아남는 이유

이 학기 내내 등장할 거의 모든 “탐험” 메커니즘 — ϵ -greedy(2.1), UCB(2.2), Ch. 6에서 Q-table 위에 다시 씌워질 ϵ -greedy — 는 사실 **정책** $\pi(a|s)$ 의 특수한 경우.

- 밴딧에서는 상태가 하나뿐이라 $\pi(a|s^*)$ 는 “ k 개 팔 중 어떤 팔을 고를지”라는 **한 줄 규칙**으로 축소.
- Ch. 6에서 Q-table이 (s, a) 행렬이 되는 순간, ϵ -greedy의 코드는 **한 줄도 바뀌지 않고** 그대로 작동 — 왜냐하면 그 코드가 **처음부터 “1-상태 MDP의 정책”**이었기 때문.

이 절에서 “같은 코드”를 확인해두면, Ch. 6이 오는 날 낯설지 않음.

벨만방정식까지: 1-상태 MDP의 가치

Ch. 3의 가치함수

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s \right]$$

를 1-상태 MDP의 유일한 상태 s^* 에 대입. 정책 π 가 팔 a 를 (확률 1로) 골라준다면, 매 스텝 보상 $R(s^*, a) = q^*(a)$ 반복:

$$V^\pi(s^*) = q^*(a) + \gamma q^*(a) + \gamma^2 q^*(a) + \cdots = \frac{q^*(a)}{1 - \gamma}$$

1-상태 MDP의 벨만방정식 $V^\pi(s^*) = R(s^*, \pi(s^*)) + \gamma \sum_{s'} P(s'|s^*, \pi(s^*)) V^\pi(s')$ 를, 전이 확률이 $P(s^*|s^*, a) = 1$ 뿐이라 줄인 형태. **기하급수**라 닫힌해가 바로 $q^*(a)/(1 - \gamma)$.

γ 가 “보는 눈의 깊이”를 바꾼다

- $\gamma = 0$: $V^\pi(s^*) = q^*(a)$ — “이 스텝의 보상만 고려하라”는 극단. 밴딧의 “이번 당김 평균 보상”과 정확히 일치.
- $\gamma \rightarrow 1$ 에 가까울수록 ($1/(1 - \gamma)$ 가 커지므로) “끝까지 계속 이 팔을 당길 미래의 총 보상”이 커짐 — “한 번의 보상”이 “무한히 반복된 총 보상”으로 확장.

예제: 진짜 최선의 팔($q^*(a^*) = 2.0$)을 영원히 고르는 정책 π^* :

- $\gamma = 0$: $V^{\pi^*}(s^*) = \mathbf{2.0}$
- $\gamma = 0.9$: $V^{\pi^*}(s^*) = 2.0/(1 - 0.9) = \mathbf{20.0}$

핵심

이 수식 하나가, “밴딧의 $q^*(a)$ ”과 “MDP의 $V^\pi(s)$ ”이 실은 같은 양의 두 극한임을 보임.

손으로 한 번: 2-상태 MDP에서 무엇이 달라지는가

가장 작은 “진짜” MDP — 상태 s_0, s_1 두 개, 각 상태에서 행동을 하나만 가지는 장난감 환경:

- s_0 : 어떤 행동을 해도 **보상 1.0**, 항상 s_1 로 전이 ($P(s_1|s_0) = 1$).
- s_1 : 어떤 행동을 해도 **보상 3.0**, 항상 **자기 자신** s_1 로 전이 ($P(s_1|s_1) = 1$) — 한 번 s_1 에 도착하면 **영원히** s_1 에 남음.

할인율 $\gamma = 0.9$ 라 하자:

- s_1 (자기 전이만): $V^\pi(s_1) = 3.0 + 0.9 V^\pi(s_1) \Rightarrow V^\pi(s_1) = 3.0/(1 - 0.9) = \mathbf{30.0}$
- s_0 (보상 1.0 한 번 $\rightarrow s_1$): $V^\pi(s_0) = 1.0 + 0.9 \cdot 30.0 = \mathbf{28.0}$

“상태 간 참조”: 밴딧에 없던 구조

- $V(s_1) = 30$: “이 상태에서 시작하면, 영원히 3.0을 할인해서 받은 총 보상”.
- $V(s_0) = 28$: “현재 s_0 에 있지만, 다음 스텝에 s_1 로 넘어가 3.0을 영원히 받기 때문에, $1.0 + 0.9 \times 30 = 28$ ”.
- **두 상태의 가치가 서로를 참조**하고 있는 모습.
- **밴딧에는** “현재 상태의 가치는 다른 상태의 가치에 달려 있다”는 구조가 **전혀 없음** — 각 팔의 가치 $q^*(a)$ 는 그 팔 자신의 평균 보상만으로 완전히 결정.
- **MDP에서는 상태 간 참조**가 생기고, 이 참조가 벨만방정식의 **재귀 구조**(Ch. 3~4 전체)를 만듦.

Ch. 4의 반복적 정책평가 한 스텝 = $V_{k+1}(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V_k(s')$ — 지금 손으로 한 $V(s_0) = 1.0 + 0.9 \cdot V(s_1)$ 이 바로 그 공식의 1-상태 전이판.

“어떤 상태가 좋은가”는 전이에 따라 바뀐다

더 중요한 관찰: 두 상태의 가치가 거의 같은(28 vs 30) 이유를 바꿔보자.
만약 s_0 의 보상을 1.0에서 **5.0으로** 높이면:

$$V(s_0) = 5.0 + 0.9 \cdot 30 = \mathbf{32.0} > V(s_1) = 30$$

→ “현재 s_0 에 있는 것이 더 좋은 상태”가 됨.

핵심 감각

“어떤 상태가 좋은가”는 그 상태 자체의 보상만으로 정해지는 게 아니라, 그 상태가 어디로 전이하는지에 따라 바뀐다.

이 “전이가 가치를 바꾼다”는 감각은 Ch. 4에서 동적계획법으로 “가치함수를 실제로 반복 계산하는 법”을 배울 때 매일 맞닥뜨릴 핵심.

확인 문제: 밴딧인가, MDP인가?

판정 기준: “**지금 한 행동이, 다음에 내가 무엇을 할 수 있는지(또는 무엇을 마주하는지) 자체를 바꾸는가?**” 바꾸지 않으면 **밴딧**, 바꾸면 **MDP**.

	시나리오	답
(a)	뉴스 기사 제목 추천 — 제목을 보여줬다고 다음 후보 가 달라지지 않음	밴딧
(b)	체스 다음 수 고르기 — 지금 둔 수가 판 배치(상태)를 바꿔 이후 선택지를 바꿈	MDP
(c)	광고 배너 노출 — 어떤 배너를 보여줬다고 다음 방문자	밴딧

밴딧과 MDP, 한 줄씩 비교

	밴딧 (Ch. 2)	MDP (Ch. 3~)
상태 $\mathcal{S}$	1개뿐 (s^*)	$ \mathcal{S} \geq 1$ 개, 보통 2개 이상
전이 $P(s' s, a)$	자기 전이뿐: $P(s^* s^*, a) = 1$	$s \rightarrow s' (s' \neq s)$ 전이 존재
가치 $V(s)$	$q^*(a)/(1-\gamma)$, 팔 하나당 하나	상태마다 다름, 상태 간 참조
최적 목표	가장 좋은 팔 1개 찾기	가장 좋은 정책 (상태마다 행동) 찾기
탐험의 대상 “지금”이 바꾸는 것	“아직 안 당겨본 팔” 아무것도 (다음도 s^*)	“아직 안 가본 상태-행동 쌍” 다음 상태 (= 미래 선택지)
이 장의 전략	ϵ -greedy, 낙관적 초기화, UCB	그대로 적용(2.1~2.2 코드가 Ch. 6

탐험 공간의 확장: k 개 팔 $\rightarrow |\mathcal{S}| \times |\mathcal{A}|$ 개 쌍

- Ch. 2: “아직 안 **당겨본 팔**”에 대한 불확실성만 처리.
- Ch. 3부터: “아직 안 **가본 상태-행동 쌍**”에 대한 불확실성까지 — 탐험해야 할 공간이 k 개 팔에서 $|\mathcal{S}| \times |\mathcal{A}|$ 개 쌍으로 **커짐**.

이 “탐험 공간의 확장”이, 왜 서로 다른 알고리즘으로 나뉘는지 설명:

- Ch. 4 **동적계획법**: “모든 상태-행동을 한 바퀴 훑는” 정책반복 — **모든 상태를 한 번에 볼 수 있으므로** 모든 쌍을 계산.
- Ch. 5~6 **몬테카를로·TD**: 에이전트가 **직접 가본** 상태-행동만 갱신.

이 구분이 왜 중요한지는 Ch. 4~6에서 매일 다시 등장.

자주 하는 실수: “상태가 하나뿐이니 MDP도 필요 없다”

“지금까지의 관찰이, 앞으로의 가능성을 바꿀 수 있는 “숨겨진” 요소가 정말로 없나?”

예: 비정상 환경 — 팔 B의 진짜 평균이 “처음 1000번은 1.0, 그 이후로는 3.0”으로 **한 번 바뀌는** 환경.

- 이 문제는 “지금의 팔 선택이 다음 상태를 바꾼다”는 뜻의 MDP는 **아님** — 형식적으로는 여전히 1-상태 MDP(= 밴딧).
- $1/N_t(a)$ 로 갱신하는 **ϵ -greedy**: “지금까지 모든 보상의 평균”을 추적 $\rightarrow$ 1.0의 기록이 오래돼서 3.0으로 바뀐 뒤에도 추정치가 **천천히만** 따라옴.
- 고정 α 로 갱신: 최근 관찰에 더 큰 가중치 $\rightarrow$ 평균이 바뀐 순간 **빠르게** 따라옴.

“전이” vs “비정상성”: 서로 다른 층

전이 (Ch. 3의 P): 구조적. 지금의 행동이 “어디로 넘어가는지”($P(s'|s, a)$)를 결정 — 세계의 **도형**이 바뀐.

비정상성 (2.1 FAQ): 분포적. 같은 상태에서 같은 행동을 해도, 그 행동이 내는 **보상의 분포**가 시간에 따라 바뀐 — 세계의 **도형은 그대로**인데 “그 도형 위에서 받는 점수”가 바뀐.

두 가지 오분류

- (i) 전이가 실제로 존재하는 문제(체스, 로봇, 환자 치료 경로)를 “밴딧으로 충분”하다고 **오분류**.
- (ii) 보상이 바뀌는 문제를 “이미 MDP를 고려했다”고 **오분류**.

둘 다 “지금의 선택이 앞으로의 가능성을 바꾸는가?”라는 **같은 질문의 서로 다른 층**에서 생기는 실수.

실습: 밴딧 → 1-상태 → 2-상태 MDP

- **A. 밴딧(=1-상태 MDP)에서 ϵ -greedy 실행:** 2.1절 코드 그대로. 동일 시드(0)에서 $\epsilon = 0.1$ 만 달리면, 두 줄의 “후회”가 **약 20배** 차이: $\epsilon = 0.1$ 은 최선 팔을 1800번 넘게 골라 총보상 ≈ 3900 , $\epsilon = 0.0$ 은 단 한 번의 불운으로 1999번 최악의 팔만 당겨 총보상 < 2000 (후회 ≈ 2000).
- **B. 1-상태 MDP의 벨만방정식 직접 대입:** $V = q^* + \gamma \cdot V$ 반복 대입. $\gamma = 0$: $V = 2.0$, $\gamma = 0.9$: $V = 20.0$ — “ γ 가 보는 눈의 깊이”.
- **C. 2-상태 MDP에서 벨만방정식 직접 풀기:** 반복 대입 $\rightarrow V(s_1) = 30.0, V(s_0) = 28.0$ — 손계산과 정확히 일치.

Ch. 4의 `policy_evaluation` 함수의 2-상태 특수판을, 지금 이 지점에서 한 번 먼저 맛봄.

자주 묻는 질문: “1-상태 MDP = 밴딧”이 동어반복인가?

Q: “상태가 하나뿐인 MDP”와 “밴딧”은 정말 수학적으로 같은 문제인가? “MDP”라 부를 추가가 실제로 있는가?

있다 — 전이 확률 $P(s'|s, a)$ 의 형태가 다름.

- 1-상태 MDP: 전이가 자기 전이뿐 $\rightarrow V(s^*) = q^*(a)/(1 - \gamma)$ 라는 **단일** 방정식으로 닫힘.
- 상태 2개 이상: $V(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s))V(s')$ 가 $|\mathcal{S}|$ 개의 **연립 방정식** — Ch. 4 “반복적 정책평가”가 풀어야 할 대상.

즉 “1-상태 MDP = 밴딧”은 **방정식의 형태가 단순화된 특수 케이스**라는 뜻이지, “아무것도 없는 동어반복”이 아님.

자주 묻는 질문: 실전에서 모델링 고르기

Q: 밴딧과 MDP 중 어느 쪽으로 모델링할지 헷갈리면?

확인 문제의 기준을 그대로: “행동이 다음 상황 **자체**를 바꾸는가?”

- 헷갈리면 일단 **밴딧으로 단순화해서 시작**하는 것도 실전에서 흔한 전략.
- 상태 전이를 무시할 수 있을 만큼 약하면 밴딧 알고리즘만으로도 충분히 좋은 성능.
- 정말 전이가 중요하다는 것이 드러나면 그때 MDP로 확장 — Ch. 3부터 이 확장을 정확히 어떻게 하는지 다룸.

Q: “정책(policy)”이라는 말이 왜 자연스러워지는가?

MDP에서 정책 = “상태마다 어떤 행동을 고를지”를 정하는 $\pi(a|s)$. 1-상태 MDP에서는 “ k 개 팔 중 하나를 고르는 **한 줄 규칙**”으로 축소 — 2.1~2.2절의 ϵ -greedy, UCB, 낙관적 초기화가 **그 한 줄 규칙**. 따라서 “밴딧의 전략”과 “1-상태 MDP의 정책”은 **같은 것을 가리키는 두 이름**.

자주 묻는 질문: 전이 vs 비정상성, 또 다시

Q: “지금의 선택이 앞으로의 가능성을 바꾼다”(MDP) 와 “보상이 시간에 따라 바뀐다”(비정상성)는 어떻게 다른가?

둘은 **서로 다른 층**의 문제.

- MDP의 전이는 **구조적**: 지금의 행동이 “어디로 넘어가는지”($P(s'|s, a)$)를 결정 — 세계의 **도형**이 바뀐.
- 비정상성은 **분포적**: 같은 상태에서 같은 행동을 해도, 그 행동이 내는 **보상의 분포**가 시간에 따라 바뀐 — 세계의 **도형은 그대로**인데 “그 도형 위에서 받는 점수”가 바뀐.

이 구분을 모르면, (i) 전이가 실제로 존재하는 문제를 “밴딧 으로 충분”하다고, (ii) 보상이 바뀌는 문제를 “이미 MDP를 고려했다”고 **오분류** — 바로 그 함정.

이번 주차 정리

- ① **밴딧은 강화학습의 가장 작은 뼈대** — 상태도 전이도 없고, “탐험-활용의 딜레마” 하나만 순수하게 남김. 총보상과 후회 $\mathcal{R}_T = Tq^*(a^*) - \sum R_t$ 라는 두 잣대.
- ② **세 가지 탐험 전략, 동작 방식이 다름** — ϵ -greedy(탐험의 양을 고정 확률로 조종), 낙관적 초기화(탐험을 초기값 하나로 예약), UCB(탐험을 매 스텝 불확실성에 비례해 재계산).
- ③ **밴딧 = 1-상태 MDP** — “상태가 하나뿐인 MDP”로 다시 읽으면, 2.1~2.2절 전략이 “정책”으로 해석되고, 두 상태만 추가돼도 가치가 **연립 방정식**이 됨.

다음 주 — Chapter 3. MDP (Markov Decision Process)

바로 이 밴딧에 “**상태**”라는 재료를 하나 더해서, 강화학습 문제를 수학적으로 엄밀하게 정의하는 틀을 세운다. 다섯 요소 ($\mathcal{S}, \mathcal{A}, P, R, \gamma$), 가치함수, 벨만방정식 — 이 장에서 기른 “탐험-활용” 직감이 그 출발점.